



Grado en Ingeniería Informática

Estructuras de Datos Avanzadas

Examen práctico de casos de uso Convocatoria ordinaria 17 de enero de 2026

Tiempo máximo: 1 hora

Instrucciones:

- La duración del examen es de 60 minutos.
- La puntuación de cada ejercicio se indica junto al enunciado.
- No se responderán preguntas sobre los enunciados. La interpretación de estos forma parte del examen.
- No está permitido el uso de calculadoras, ni la consulta de libros, apuntes o dispositivos electrónicos de ningún tipo durante el examen.
- Desde el Aula Virtual hay que descargar un archivo ZIP que contiene el esqueleto de los ejercicios del examen y las pruebas unitarias asociadas.
- El estudiante debe generar un fichero ZIP con el directorio en el que esté el código desarrollado durante el examen y entregarlo en la actividad “Examen práctico” disponible en Aula Virtual.

Consideraciones adicionales:

- No se pueden añadir nuevas clases, métodos o propiedades públicas.
- Solo se pueden añadir nuevas clases si son privadas. A las clases ya dadas solo se les puede añadir métodos y propiedades con visibilidad privada.
- No se pueden modificar las cabeceras de los métodos ya dados.
- El código del proyecto proporcionado se ha probado utilizando **Oracle OpenJDK 24.0.2** (disponible en el Escritorio de Desarrollo de MyApps) y **JUnit 5.8.1**.
- Para poder evaluar la implementación de un método, es necesario que las pruebas unitarias asociadas a este se ejecuten correctamente, mostrando el comportamiento esperado.

Enunciado

El sistema autónomo 28766 que pertenece al ISP URJCNetServices es una red formada por routers y enlaces entre routers. En este sistema, los routers intercambian mensajes entre sí mediante los enlaces. Algunos de estos mensajes están marcados como *mensajes de difusión* y deben ser propagados por toda la red. Cuando un router recibe un mensaje de difusión, debe reenviarlo inmediatamente a todos los routers con los que esté conectado, excepto a aquel desde el que lo recibió. Sin embargo, la red debe evitar que un mismo mensaje circule indefinidamente provocando tormentas de difusión.

Para implementar estas funcionalidades en el futuro, se va a proveer a los routers de memoria. Es decir, se quiere que los routers almacenen los mensajes de difusión que han recibido.

Cada mensaje de difusión contiene la siguiente información:

- Un identificador único.
- La hora en la que fue enviado.
- La dirección IP del router que lo envió.

Cuando un router recibe un mensaje de difusión debe verificar si ya lo había recibido antes o si es la primera vez que lo recibe.

Además, URJCNetServices quiere seleccionar uno de los routers para que este sea el encargado de comenzar la difusión de mensajes. Para ello, quiere ser capaz de encontrar el router más cercano en general a todos los otros routers de la red. La forma de resolver esto es mediante la búsqueda del árbol de difusión mínimo de la red. Un árbol de difusión es un árbol que contiene todos los nodos de la red y el mínimo número de enlaces de la red necesario para que todos los nodos estén conectados en el árbol. A continuación, se incluyen varias figuras a modo de ejemplo. La Figura 1 contiene una red de ejemplo con 6 routers y enlaces entre ellos. La Figura 2, la Figura 3 y la Figura 4 contienen el árbol de difusión obtenido a partir de la red mencionada anteriormente y los routers 1, 3 y 4, respectivamente.

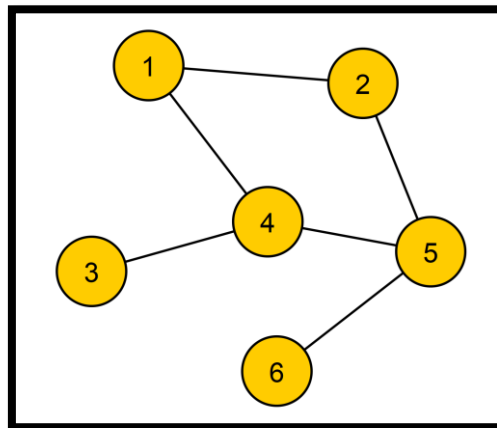


Figura 1. Red con routers y enlaces.

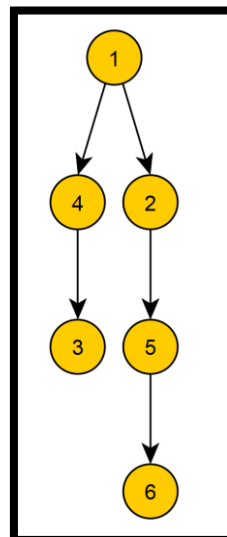


Figura 2. Árbol de difusión a partir del router 1.

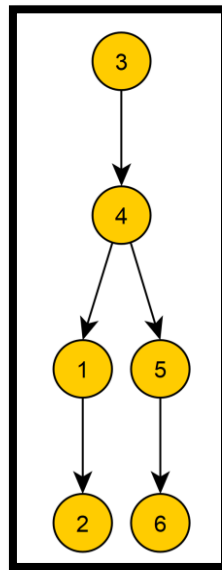


Figura 3. Árbol de difusión a partir del router 3.

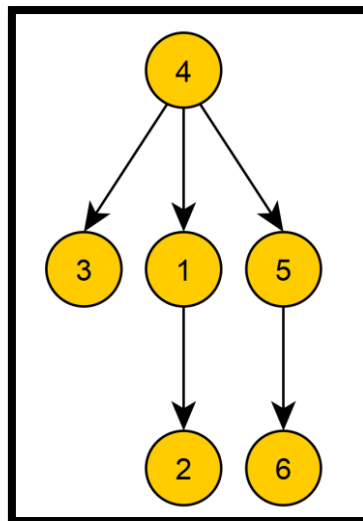


Figura 4. Árbol de difusión a partir del router 4.

En la clase **DateComparator** se pide:

1. [1 punto] Implementar el método **compare()**.

En la clase **Router** se pide:

1. [0,25 puntos] Seleccionar la o las estructuras de datos adecuadas para la implementación de la clase.
2. [1 punto] Implementar el método **mensajeDeDifusion()**, que recibe un mensaje de difusión. Si el mensaje no se había recibido anteriormente, el método devolverá *true*. En caso contrario, devolverá *false*. Si el mensaje es null, se debe lanzar la excepción *IllegalArgumentExpection*.
3. [2,5 puntos] Implementar el método **borrarMensajesAntiguos()**. Este método recibe una fecha y elimina todos los mensajes con fecha igual o anterior. Devuelve el número de mensajes que han sido borrados de la memoria del router.

En la clase **NetServices** se pide:

1. [0,25 puntos] Seleccionar la o las estructuras de datos adecuadas para la implementación de la clase NetServices.
2. [1 puntos] Implementar el método **addRouter()**. Este método añade un router a la red. Si el router ya existe en la red, el método devolverá *false*. Si el router no existe en la red, se añadirá y el método devolverá *true*.
3. [1 puntos] Implementar el método **addConnection()**. Este método añade una conexión entre dos routers. Si alguno de los routers no existe en la red, el método devolverá *false*. Si ambos routers existen, se añadirá la conexión y el método devolverá *true*.
4. [3 puntos] Implementar el método **centralNode()**. Este método calcula el nodo central de la red. El nodo central es aquel que minimiza la distancia máxima a cualquier otro nodo de la red. Es decir, tras calcular el árbol de expansión mínimo para la red, el nodo central es la raíz de dicho árbol. Si la red no tiene routers, se lanzará una excepción. Si la red no es conexa, se devolverá *null*.